

Regular Expressions

Natural Language Processing

Daniel Ortiz Martínez

Table of Contents

1. Introduction
2. Building Regular Expressions with Python

Introduction

Why do we need regular expressions?

Imagine you need to process thousands of text files to:

- Extract all email addresses
- Validate phone numbers in different formats
- Find and replace specific patterns
- Verify that passwords meet certain requirements

Why do we need regular expressions?

Imagine you need to process thousands of text files to:

- Extract all email addresses
- Validate phone numbers in different formats
- Find and replace specific patterns
- Verify that passwords meet certain requirements

Doing this manually would be tedious and inefficient!

Why do we need regular expressions?

Imagine you need to process thousands of text files to:

- Extract all email addresses
- Validate phone numbers in different formats
- Find and replace specific patterns
- Verify that passwords meet certain requirements

Doing this manually would be tedious and inefficient!

Regular expressions allow us to describe these patterns concisely and process them automatically

What is a Regular Expression?

A **regular expression** (regex or regexp) is a sequence of characters that defines a *search pattern* in text strings

Main characteristics:

- Formal language for describing text patterns
- Uses literal characters and special metacharacters
- Supported by most programming languages
- Based on automata theory and formal languages

What is a Regular Expression?

A **regular expression** (regex or regexp) is a sequence of characters that defines a *search pattern* in text strings

Main characteristics:

- Formal language for describing text patterns
- Uses literal characters and special metacharacters
- Supported by most programming languages
- Based on automata theory and formal languages

In essence: *a compact way to express “all texts that have this structure”*

Motivating Example

Problem: Validate email addresses

Motivating Example

Problem: Validate email addresses

Without regular expressions:

- Check that there is exactly one @ symbol
- Verify there is text before and after the @
- Verify there is at least one dot after the @
- Check for allowed characters...
- Dozens of lines of code!

Motivating Example

Problem: Validate email addresses

Without regular expressions:

- Check that there is exactly one @ symbol
- Verify there is text before and after the @
- Verify there is at least one dot after the @
- Check for allowed characters...
- Dozens of lines of code!

With a regular expression:

```
[^@]+@[^@]+\.[^@]+
```

Building Regular Expressions with Python

Generate a regular expression using `p = re.compile(r'regex')`

- `p` will be a `re.Pattern` object that we can apply to any string
- `p.findall(s)` returns a list containing all substrings within `s` that satisfy our regular expression
- `p.finditer(s)` returns a generator. Each element of the generator is an `SRE_Match` object which contains:
 - `span(x,y)` indicates the start position `x` and end position `y` (of `s`)
 - `match= '...'` indicates the string that satisfies the regular expression

Special Regex Characters

- `'.'` Matches any character except new line.
- `'d'` Matches any digit (0-9). Equivalent to `'[0-9]'`
- `'D'` Matches any NON digit. Equivalent to `'[^0-9]'`
- `'w'` Matches any "word character". Equivalent to `'[a-zA-Z0-9_]'`
- `'W'` Matches any NON alphanumeric character
 - Equivalent to `'[^a-zA-Z0-9_]'`
- `'s'` Matches any whitespace (space, tab, newline)
 - Equivalent to `'[\t\n\r\f\v]'`
- `'S'` Matches any NON whitespace (space, tab, newline)
 - Equivalent to `'[^ \t\n\r\f\v]'`

Quantifiers

- Quantifiers are operators that are applied to the preceding symbol
- '*' previous symbol appears 0 or more times
- '+' previous symbol appears 1 or more times
- '?' previous symbol appears at most one (0 or 1)
- '{k}' previous symbol repeated k exact times
- '{min,max}' previous symbol appears between min and max times
 - For example '{2,8}' would match numbers between 2 and 8

Quantifiers

- `"r'a.*'"` the `'*'` refers to `'.'` making this expression get triggered when `'a'` is followed by any character
- `"r'd{4}'"` the `'{4}'` refers to `'d'`, making this regular expression get triggered with 4 consecutive digits
- Example:

```
aux="The girl who loved the cat ended up with a catwomen costume"
p = re.compile(r'cat\s')
p.findall(aux)
['cat ']
p = re.compile(r'cat\s*')
p.findall(aux)
['cat ', 'cat']
```


Regex Example

- Let us consider the following example:

```
aux="The girl who loved the cat ended up with a catwomen costume"
p = re.compile(r'cat')
p.findall(aux)
['cat', 'cat']
p = re.compile(r'cat\s')
p.findall(aux)
['cat ']
p = re.compile(r'cat.*')
p.findall(aux)
['cat ended up with a catwomen costume']
```

- `'\b'` Matches any word boundary
- `'\B'` Matches any NON word boundary
- `'^'` Matches beginning of a string
- `'[^a-e]'` negates the character set `'a-e'`
- `'$'` Matches a position that is end of a string

Character Sets

- `'[]'` allows us to specify sets of symbols
- `'[ab3-]'` matches any character `'a'` or `'b'` or `'3'` or `'-'`
- `'[a-g]'` matches any character `'a'` to `'g'` such as `'b'`, `'c'`, ..., `'g'`
- `'[A-G]'` matches any character `'A'` to `'G'` such as `'B'`, `'C'`, ..., `'G'`
- `'[a-zA-G]'` matches any lowercase character and any uppercase character from `'A'` to `'G'`
- `'[d1-d2]'` matches any digit between `'d1'` and `'d2'`. For example `'[0-5]'` matches `'0'`, `'1'`, `'2'`, `'3'`, `'4'`, `'5'`

Examples

- `abc*` matches a string that has `ab` followed by zero or more `c`
- `abc+` matches a string that has `ab` followed by one or more `c`
- `abc?` matches a string that has `ab` followed by zero or one `c`
- `abc{2}` matches a string that has `ab` followed by 2 `c`
- `abc{2,}` matches a string that has `ab` followed by 2 or more `c`
- `abc{2,5}` matches a string that has `ab` followed by 2 up to 5 `c`
- `a(bc)*` matches a string that has `a` followed by zero or more copies of the sequence `bc`
- `a(bc){2,5}` matches a string that has `a` followed by 2 up to 5 copies of the sequence `bc`